Hong Kong Metropolitan University

COMP2090SEF

Data Structures, Algorithms and Problem Solving

Course Project Report

Group 36

Submission date: 4/10/2026

We declare that:

(i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged.

(ii) the project, in parts or as a whole, has not been submitted for more than one purpose without declaration.

(iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequences when breaching such policy and regulations.

(iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and

(v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
XIA Weihao	14167751
SITU Ruiyuan	14186925
Shi Zhenghang	14058130

This report is for TASK 1

1.1 Problem definition and background

Urban fire emergencies, like the 2025 Tai Po Market blaze, reveal critical gaps in real-time decision-making and adaptability to dynamic conditions such as weather conditions. The incident revealed three critical issues:

- ✧ **Delayed situational awareness** - slow dispatch of rescue to high-risk zones.
- ✧ **Ineffective resource coordination** - firetrucks dispatched without prioritization.
- ✧ **Poor adaptability to environmental dynamics** – Failed to adjust fire spread models in real time based on wind speed, rainfall, or other factors.

This project addresses these challenges by developing an interactive OOP simulation system that enhances firefighting efficiency through dynamic visualization and scenario testing.

1.2 Survey of Existing Solutions in Practice

- ✧ **Manual command systems** (e.g., fire department dispatch centers) - Rely on human experience but lack real-time data integration or dynamic simulation. (McBurney, 2020)
- ✧ **GIS-based planning tools** - fail to simulate dynamic fire behavior or unit interactions (Karatas et al., 2025).
- ✧ **Commercial simulation software** (e.g., FDS + Evac) - offers advanced capabilities but is closed-source, expensive, and computationally complex, making it unsuitable for frontline training (Shin et al., 2024).

1.3 Data Structures and Their Usage in the Project

The system's data structures are **modular OOP components** distributed across the project's core modules. Each structure serves a specific purpose in simulating fire dynamics and rescue operations. Folder structure details shown in (Fig. 1)

- **Core Layer (*core/*)**
- ✧ **world.py** - Maintains a **2D grid of cell objects** representing the city map. Each cell store: Terrain type (road, building, non-flammable), Fire status (*intensity*, burning/non-burning), Weather impact (wind, rain, sunny), Occupying units (e.g., FireTruck).
- ✧ **entities.py** - Defines **polymorphic entity classes** (Fire, Emergency Unit,), where each instance encapsulates: Position, status attributes (e.g., *fire intensity*), Behavior methods (e.g., ignite).
- ✧ **pathfinding.py** - Implements the ***A star*** algorithm to compute optimal paths
- ✧ **events.py** - Manages an **event queue** (e.g., *FireSpreadEvent*, *UnitDispatchEvent*) to trigger state updates (e.g., *fire intensity* changes, unit movement) in a time-stepped simulation.
- **Render & UI Layers (*render/, ui/*)**

- ✧ **renderer.py** - Reads map and entity states to render the visual simulation (e.g., *fire intensity colors*, unit positions).
- ✧ **panels.py** - Provides interactive controls (e.g., "Ignite Fire", "Toggle Weather")

This project isn't based on the previous work from Comp1080SEF

2.Program functions and Problem solutions

- The Hong Kong-based map features a balanced grid layout with high-rise buildings and dense population. (Fig. 2)
- A simple control panel and keyboard shortcuts enhance usability. (Figs. 3, 4)
- Weather and speed settings support realistic simulation. (Fig. 5)
- Statistics, event log, and inspectors improve result visualization. (Figs. 6–8)
- ◆ **Problem Solution:**

  We assume there are 3 fire stations placed in **realistic locations around the city**. When you click the demo button, a fire starts at a random spot, just like a real emergency. The fire spread speed and direction is **based on the weather.** For example, **strong winds will push the flames faster in one direction, while rain will slow it down.**

  To fight the fire, the system automatically sends the nearest firetruck from the closest station. It uses the *A star* algorithm to find the absolute best route, considering not only the city's road layout but also **how intense and fast the fire is spreading nearby**. By **factoring in both distance and the urgency of the fire**, the algorithm can **dynamically prioritize paths** to ensure the quickest and most effective response. This improves on the old system where a human might just send the closest truck without realizing that the fire is getting worse and perhaps choose a less suitable route. A control panel ensuring the rescue supervisor **watches everything happening in real time**—they can see the fire's size, how long the **fire existed** and exactly how long **the response took**. This gives them a clear, data-driven picture, once the fire is out, the program **automatically stops.** This complete cycle turns a simple simulation **into a powerful training tool.**

	Test Case1	Test Case2
Set up	Rain	Wind
action	2xspeed+demo	8xspeed+demo
result	Figure 9	Figure 10

3.Challenges, Reflections, and Future Work

3.1 Overcoming Key Challenges :

Code Complexity in Multi-Agent Coordination

Early development had tightly coupled logic between fire, units, and weather, complicating debugging. We used the **Observer pattern** to decouple modules, letting Weather updates trigger independent reactions in Fire and Emergency Unit without

messy loops.

Defining Realistic Rescue Prioritization

Simple "nearest target" logic ignored fire severity, causing unrealistic behavior. We researched **NFPA triage protocols** to build a weighted model for prioritizing targets, ensuring units tackle high-intensity threats first, then using *A star* for optimal navigation.

3.2 Reflections:

Initially we thought this project was just about animating fire spread and deploying fire trucks. Soon we realized it required real-time decision-making, environmental dynamics, and multi-agent coordination, far beyond a visual demo. As our first OOP project, it improved our ability to design modular architecture and apply encapsulation. At first, rescue priority seemed simple until we saw unrealistic and unreasonable unit behaviors. Meanwhile, researching NFPA protocols and adding a weighted model with the A star algorithm taught us this was not just coding, but turning real-world tactics into logic. With help from professionals, we became more skilled at using AI tools, giving precise prompts to scaffold code and debug. We acknowledged our current poor programming skills, but we are confident that further study at HKMU will help us master more techniques and improve our problem-solving and debugging skills.

3.3 Future Enhancements ：

We will further utilize high-resolution GIS data and real-time traffic sources for accurate and dynamic urban mapping. In addition, implementing sophisticated algorithms like *D star Lite* for efficient route planning in complex environments. We will try to develop AI agents for coordinated emergency response, optimizing decision-making through reinforcement learning.

4.AI Usage Statement: Only for coding and debugging

5.Reference

Karatas, A., & Yildirim, R. (2025). Estimating fire response times and planning optimal routes using GIS and machine learning techniques. *Applied Sciences, 15*(1), 345. https://doi.org/10.3390/app15010345

McBurney, P. A. (2020). *Significant weaknesses within Northern Ireland Fire Service governance due to outdated IT systems.* Belfast Telegraph. https://www.belfasttelegraph.co.uk/news/northern-ireland/significant-weaknesses-within-fire-service-governance-due-to-outdated-it-systems-report-finds/article385547.ece

Shin, J. R., Kim, J. H., & Kang, Y. S. (2024). Comparison of parallelization performance of fire dynamic simulator (FDS) program. *Journal of Computational Fluids Engineering, 29*(4), 175–188. https://doi.org/10.1007/s10694-025-01781-3

Appendix

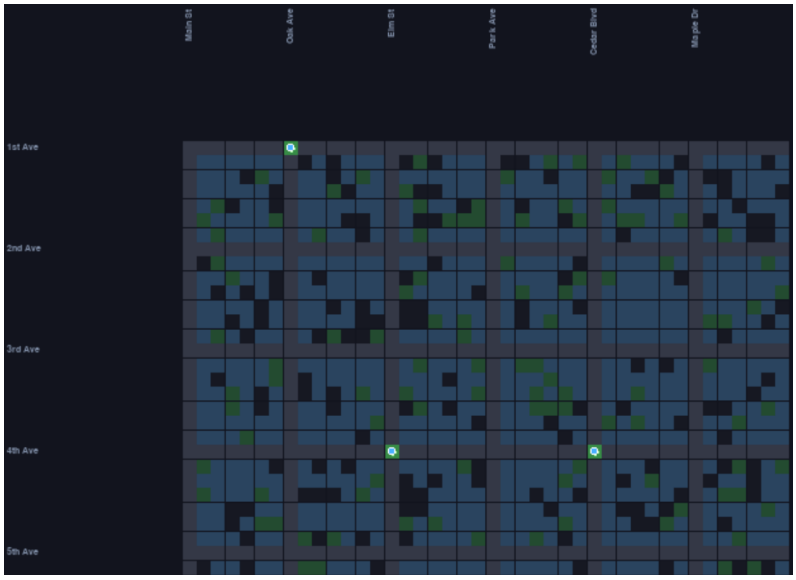
Workload

Members	Task 1
XIA Weihao(PL)	Ideas&Code&Report&Debugging
SITU Ruiyuan	Ideas&Debugging
SHI Zhenghang	Ideas&Debugging

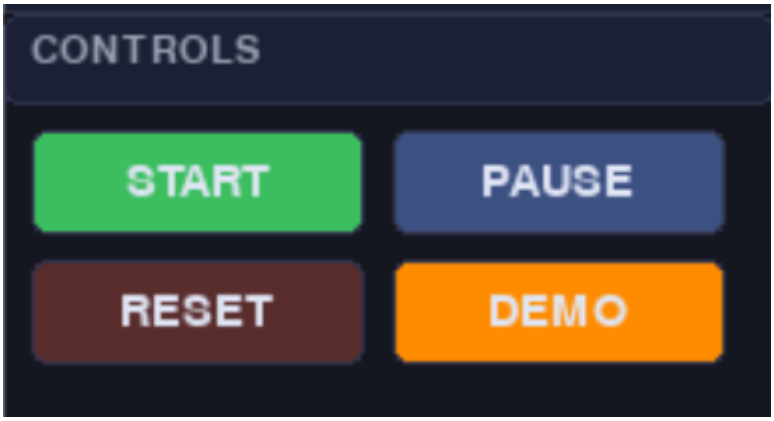
Attachments

```
src/
├── __pycache__/            # Auto-generated Python bytecode (runtime artifact)
├── core/                   # Core simulation logic
│   ├── __init__.py
│   ├── entities.py         # Game entities (Fire, Firetruck, etc.)
│   ├── events.py           # Event system for state changes
│   ├── pathfinding.py      # A* algorithm for agent navigation
│   ├── simulation.py       # Main simulation loop & update logic
│   └── world.py            # World state: map, fire spread, time, environment
├── models/                 # Data models, configs, enums, helper structures
├── render/                 # Rendering system
│   ├── __init__.py
│   ├── camera.py           # View control (pan/zoom)
│   ├── particles.py        # Particle effects (smoke, embers)
│   ├── renderer.py         # Main rendering logic (draw map, entities, UI)
│   └── theme.py            # Visual themes (colors, styles)
├── ui/                     # User interface components
│   ├── __init__.py
│   └── panels.py           # UI panels (controls, info displays)
├── visualization/          # GUI management
│   ├── gui_manager.py      # Main GUI controller
│   ├── __init__.py
│   └── DejaVuSans.ttf      # Font file for UI text
├── main.py                 # Entry point of the application
└── README.md               # Project documentation (optional)
```

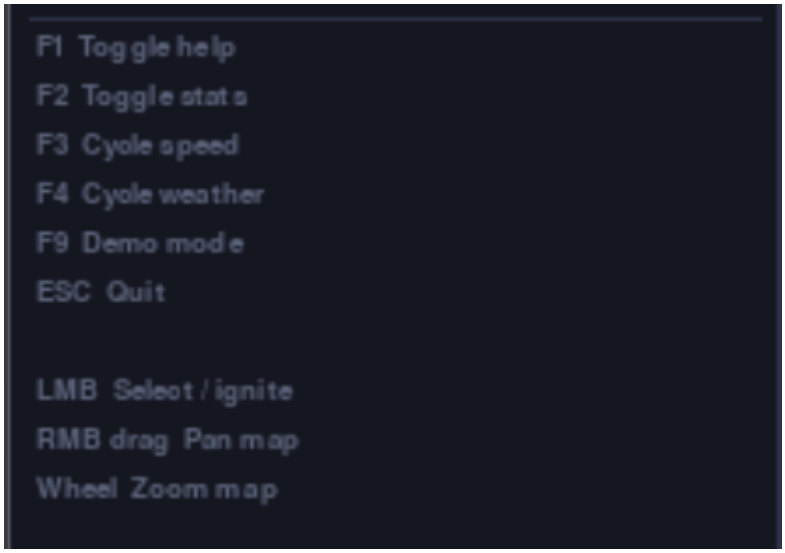
(Figure 1-Folder structure overview)



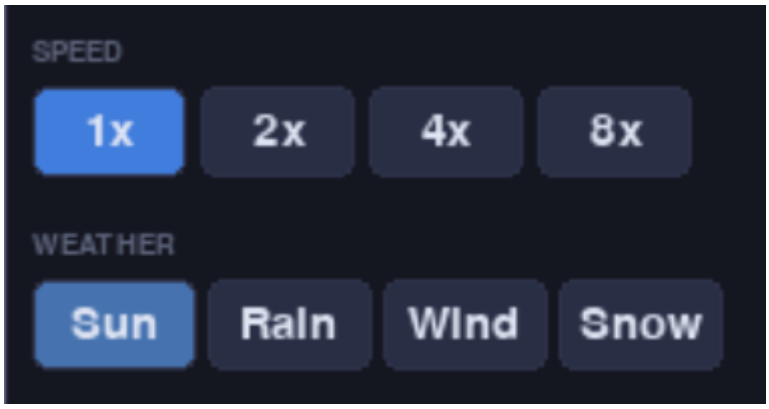
(Figure 2-The map of our project)



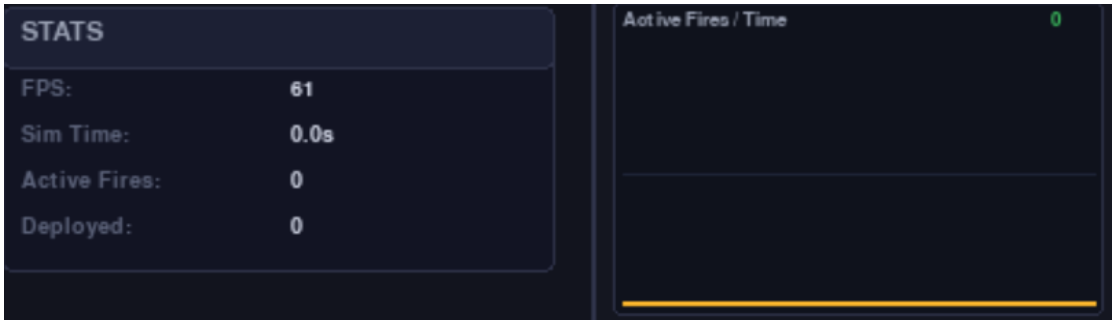
(Figure 3-The control panel)



(Figure 4-keyboard shortcuts)



(Figure 5-weather and speed condition)



(Figure 6-statistic panels)

(Figure 7-inspector panel)

(Figure 8-event log panel)

(Figure 9-2x speed with raining condition frame)

(Figure 10-8x speed with intense wind condition)